

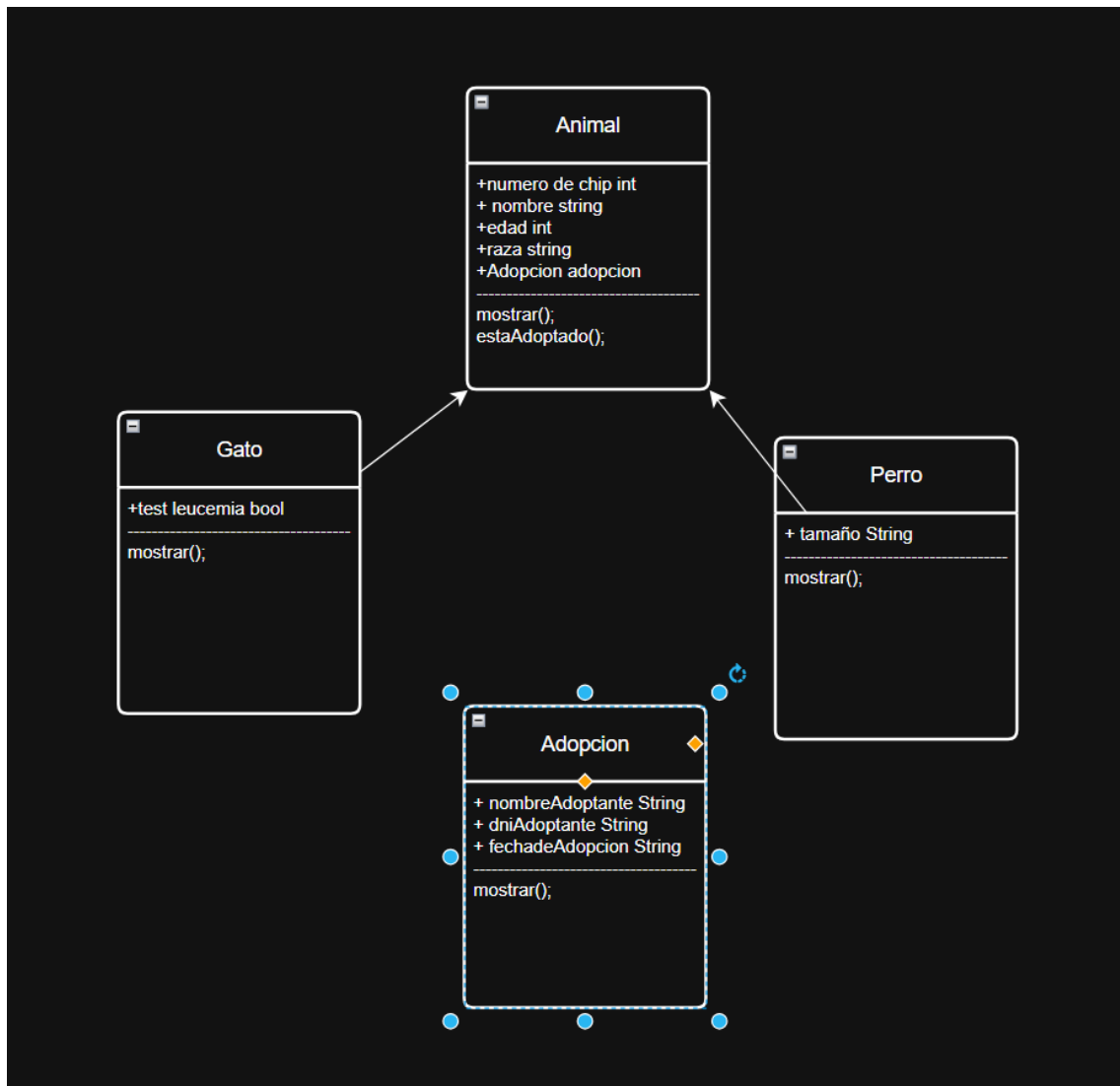
HITO 1 – PARTE 2 DEL 3º TRIMESTRE DE PROGRAMACIÓN

Alejandro Cortés Díaz

Índice

Diagrama de clases.....	2
Definicion de clases.....	2
Funcionamiento código.....	22
Enlace a GitHub	27

Diagrama de clases



Definicion de clases

Animal;

```
1
2 public abstract class Animal {
3     int numero_chip;
4     String nombre;
5     int edad;
6     String raza;
7     Adopcion adopcion;
8
9     public Animal (int numero_chip, String nombre, int edad, String raza, Adopcion adopcion) {
10
11         this.numero_chip=numero_chip;
12         this.nombre=nombre;
13         this.edad=edad;
14         this.raza=raza;
15         this.adopcion = adopcion;
16     }
17
18     public boolean estaAdoptado() {
19         return adopcion != null;
20     }
21
22     abstract void mostrar();
23 }
24
```

//Creo la clase abstracta animal de la que saldrán sus hijas, con un constructor que contiene prácticamente todos los atributos de las otras dos clases, y un método que posteriormente, cada clase tendrá que sobrescribir con sus propios datos, añado estaAdoptado para comprobar si se encuentra en dicho estado

```
public abstract class Animal {
```

```
    int numero_chip;
```

```
    String nombre;
```

```
    int edad;
```

```
    String raza;
```

```
    Adopcion adopcion;
```

```
    public Animal (int numero_chip, String nombre, int edad, String raza, Adopcion adopcion) {
```

```
        this.numero_chip=numero_chip;
```

```
        this.nombre=nombre;
```

```
        this.edad=edad;
```

```
        this.raza=raza;
```

```
        this.adopcion = adopcion;
```

```
    }
```

```
    public boolean estaAdoptado() {
```

```

        return adopcion != null;
    }
}

```

```

abstract void mostrar();
}

```

Perro;

```

public class Perro extends Animal {
    String tamaño;

    public Perro (int numero_chip, String nombre, int edad, String raza, Adopcion adopcion, String tamaño) {
        super (numero_chip, nombre, edad, raza, adopcion);
        this.tamaño= tamaño;
    }

    @Override void  mostrar(){
        System.out.println("Número de chip del perro : "+ numero_chip+"\n"+ "Nombre ; "+nombre + "\n"+ "edad: "+edad+ "de raza: "+raza+"\n" + "De tamaño:"+tamaño);
        if (estaAdoptado()) {
            adopcion.mostrar();
        } else {
            System.out.println("No está adoptado.");
        }
    }
}

```

//Creo clase Perro que procede de Animal, añade el atributo "tamaño", lo que se refleja en el constructor, y luego la sobreescritción del método mostrar. Incluyo el objeto de Animal como atributo
public class Perro extends Animal {

String tamaño;

public Perro (int numero_chip, String nombre, int edad, String raza, Adopcion adopcion, String tamaño) {

super (numero_chip, nombre, edad, raza, adopcion);

this.tamaño= tamaño;

}

@Override void mostrar(){

System.out.println("Número de chip del perro : "+ numero_chip+"\n"+ "Nombre ; "+nombre + "\n"+ "edad: "+edad+ "de raza: "+raza+"\n" + "De tamaño:"+tamaño);

if (estaAdoptado()) {

adopcion.mostrar();

} else {

System.out.println("No está adoptado.");

}

```

    }
}

```

Gato;

```

public class Gato extends Animal{
    boolean test_leucemia;

    public Gato (int numero_chip, String nombre, int edad, String raza, Adopcion adopcion, boolean test_leucemia) {
        super (numero_chip, nombre, edad, raza, adopcion);
        this.test_leucemia= test_leucemia;
    }
    @Override void mostrar(){
        System.out.println("Número de chip del gato : "+ numero_chip+"\n" + "Nombre : "+nombre + "\n" + "edad: "+edad+ "\n" + "de raza: "+raza+"\n " + "Tiene el test de leucemia?" + test_leucemia);
        if (estaAdoptado()) {
            adopcion.mostrar();
        } else {
            System.out.println("No está adoptado.");
        }
    }
}

```

//Creo clase Perro que procede de Animal, añade el atributo "test_leucemia", lo que se refleja en el constructor, y luego la sobrescripción del método mostrar. Incluyo el objeto de Animal como atributo

```

public class Gato extends Animal{

    boolean test_leucemia;

    public Gato (int numero_chip, String nombre, int edad, String raza, Adopcion adopcion,
boolean test_leucemia) {

        super (numero_chip, nombre, edad, raza, adopcion);

        this.test_leucemia= test_leucemia;

    }

    @Override void mostrar(){

        System.out.println("Número de chip del gato : "+ numero_chip+"\n" + "Nombre ;
"+ nombre + "\n" + "edad: "+ edad+ "\n" + "de raza: "+ raza+"\n " + "Tiene el test de
leucemia?" + test_leucemia);

        if (estaAdoptado()) {

            adopcion.mostrar();

        } else {

            System.out.println("No está adoptado.");

        }

    }

}

```

Main;

```
import java.util.*; // Importo las utilidades de java

public class Main {

    Scanner sc = new Scanner (System.in); // Creo la instancia del scanner

    ArrayList <Animal> animals = new ArrayList<>(); // Y el arraylist donde depositaré los objetos de la clase padre Animal

    public static void main (String[] args) { // En main creo la instancia de la clase para realizar las operaciones de los métodos

        Main hitoi = new Main();
        hitoi.menu();
    }

    public void menu () {

        try { // Try catch para añadir una corrección de errores en el caso de que no funcione correctamente o de error (aplica el mismo comentario en los otros try catch)

            int opcion;

            do { // Me asegura de que al menos una vez muestre todas las opciones pues será necesario para interactuar con el programa, hasta que la opción seleccionada sea la 4, salir

                System.out.println(" ¡Bienvenido! \n Podemos a su disposicion las siguientes interacciones: \n Posibles elecciones: \n -----");

                System.out.println("1 - Agregar perro \n 2 - Agregar gato \n 3 Listar animales \n 4 - Buscar animal \n 5 Realizar adopcion \n 6 Dar de baja animal \n 7 Mostrar estadistica");

                opcion = sc.nextInt();

                // Cada caso es una opción dentro del programa, en la tercera listo todos los animales, en la cuarta a través del scanner pido el chip y llama a la función para buscar según el
                //La quinta me permite adoptar un animal que con los atributos de la instancia de Adopcion. La sexta elimina el registro del animal pidiendo el chip o id.
                //La séptima me muestra el total de gatos adoptados y el total de tests que tienen. La octava opción permite salir del bucle while, mientras que si se da cualquier otra se señal

                switch(opcion) {

                    case 1:

                        añadirPerro();
                        break;

                    case 2:

                        añadirGato();
                        break;

                    case 3:

                        listarAnimales();
                        break;

                    case 4:

                }

            }

        }

    }

}
```

```

55         mostrarAnimals(chip);
56         break;
57
58     case 5:
59         realizarAdopcion();
60         break;
61
62     case 6:
63         darleBajaAnimal();
64         break;
65
66     case 7:
67         mostrarEstadisticasGatos();
68         break;
69
70     case 8:
71         System.out.println("saliendo del programa ...");
72         break;
73
74     default:
75         System.out.println("No es opcion válida");
76         System.out.println("La elección anterior fue " + opcion);
77     }
78
79     while (opcion != 0);
80
81 }catch(Exception e) {
82     System.out.println("Exception"+e);
83 }
84
85 }
86
87 // En esta función (dentro de catch try por los posibles errores), establezco que el número de chip sea el tamaño del array +1, lo que evita repeticiones de id, así como que sea una adición
88 // Poco a poco se piden los valores de los atributos correspondientes que se añadirán a la nueva instancia de perro con los valores coincidentes del constructor
89 // En adición creo una instancia de Adopcion para definir si esta adoptado o no, en el caso de que lo este, entonces pido los datos para luego, en la adición del perro, o del gato correspondiente
90 public void añadirPerro() {
91
92     try {
93
94         int num_chip = numAnimales()+1;

```

```

95         int num_chip = numAnimales()+1;
96
97         sc.nextLine();
98
99         System.out.println("Introduce nombre.");
100         String nombre = sc.nextLine();
101
102         System.out.println("Introduce edad.");
103         int edad = sc.nextInt();
104
105         sc.nextLine();
106
107         System.out.println("Introduce raza.");
108         String raza = sc.nextLine();
109
110         Adopcion adopcion = null;
111
112         System.out.println("es adoptado?");
113         boolean esAdoptado = sc.nextBoolean();
114
115         sc.nextLine();
116
117         if (esAdoptado) {
118             System.out.println("Nombre del adoptante:");
119             String nombreAdoptante = sc.nextLine();
120
121             System.out.println("Teléfono del adoptante:");
122             String telefono = sc.nextLine();
123
124             System.out.println("Fecha de adopción (DD/MM/YYYY):");
125             String fecha = sc.nextLine();
126
127             adopcion = new Adopcion(nombreAdoptante, telefono, fecha);
128         }
129
130         System.out.println("Introduce tamaño.");
131         String tamaño = sc.nextLine();
132
133         animals.add(new Perro(num_chip,nombre,edad,raza,adopcion,tamaño));
134     }catch(Exception e) {
135         System.out.println("Exception"+e);
136     }
137 }
138
139 // En esta función (dentro de catch try por los posibles errores), establezco que el número de chip sea el tamaño del array +1, lo que evita repeticiones de id, así como que
140 // Poco a poco se piden los valores de los atributos correspondientes que se añadirán a la nueva instancia de perro con los valores coincidentes del constructor
141 // En adición creo una instancia de Adopcion para definir si esta adoptado o no, en el caso de que lo este, entonces pido los datos para luego, en la adición del perro, o del gato correspondiente

```

```

public void añadirGato() {
    try {

        int num_chip = numAnimales()+1;
        System.out.println("Introduce nombre.");

        sc.nextLine();
        String nombre = sc.nextLine();

        System.out.println("Introduce edad.");
        int edad = sc.nextInt();

        System.out.println("Introduce raza.");
        String raza = sc.nextLine();
        sc.nextLine();

        Adopcion adopcion = null;

        System.out.println("es adoptado?");
        boolean esAdoptado = sc.nextBoolean();

        sc.nextLine();

        if (esAdoptado) {
            System.out.println("Nombre del adoptante:");
            String nombreAdoptante = sc.nextLine();

            System.out.println("Teléfono del adoptante:");
            String telefono = sc.nextLine();

            System.out.println("Fecha de adopción (DD/MM/YYYY):");
            String fecha = sc.nextLine();

            adopcion = new Adopcion(nombreAdoptante, telefono, fecha);
        }

        System.out.println("¿Tiene test leucemia?");
        boolean test_leucemia = sc.nextBoolean();

        animals.add(new Gato(num_chip,nombre,edad,raza,adopcion,test_leucemia));
    } catch (Exception e) {
        System.out.println("Excepcion"+e);
    }
}
}

```

```

// Esta es la función utilizada para obtener el tamaño del arraylist, en base a lo cual marca cuál tendría que ser el valor del array. Una variable en un bucle for recorre el array, y su val
public int numAnimales() {
    int no = 0;
    for (Animal animal : animals) {
        no ++;
    }
    return no;
}

// Esta función utiliza un bucle for para recorrer el array, en busca del mismo valor de chip que se introduce como parámetro, en el momento en el que el valor "numero_chip", coincide con el
// utiliza la función mostrar(), presente en ambas clases, para entregar los datos del animal
public void listarAnimales() {
    System.out.println("Listado de todos los animales:\n _____\n");
    for (Animal a : animals) {
        a.mostrar();
        System.out.println("_____");
    }
}

//Esta función pide los datos del animal, en el caso de que el chip del mismo no se encuentre, devuelve mensaje de error por no estar registrado, si ya existe la instancia de adopcion para e
// devuelve que ya está adoptado, de lo contrario, solicita los datos correspondientes
public void realizarAdopcion() {
    try {
        System.out.println("Introduce el número de chip del animal que será adoptado:");
        int chip = sc.nextInt();
        sc.nextLine();

        //Aquí uso la función buscarPorChip para comprobar que exista, definida mas abajo
        Animal animal = buscarPorChip(chip);
        if (animal == null) {
            System.out.println("El animal no está registrado.");
            return;
        } else if (animal.adopcion != null) {
            System.out.println("Este animal ya ha sido adoptado.");
        } else {
            System.out.println("Nombre del adoptante:");
            String nombre = sc.nextLine();

            System.out.println("Teléfono del adoptante:");
            String telefono = sc.nextLine();

            System.out.println("Fecha de adopción (DD/MM/YYYY):");
            String fecha = sc.nextLine();

```

```

    } catch (Exception e) {
        System.out.println("Excepción: " + e.getMessage());
    }
}

// Esta función pide el chip y si no existe la instancia con ese valor de chip devuelve que no está registrado, de lo contrario, si es null es decir, si existe, entonces lo elimina.
// Antes de esto, si resulta que además estaba adoptado, me aseguro de eliminar primero la instancia, dejando el objeto y los datos a null, antes de eliminarlo del todo
public void darDeBajaAnimal() {
    try {
        System.out.println("Introduce el número de chip del animal para darle baja:");
        int chip = sc.nextInt();
        sc.nextLine();

        Animal animal = buscarPorChip(chip);
        if (animal == null) {
            System.out.println("El animal no está registrado.");
            return;
        } else {
            if (animal.adoptado != null) {
                System.out.println("El animal estaba adoptado, elimino datos");
                animal.adoptado = null;
            }
            animals.remove(animal);
            System.out.println("Animal dado de baja con éxito.");
        }
    } catch (Exception e) {
        System.out.println("Excepción: " + e.getMessage());
    }
}

// Muestro las estadísticas de los gatos con un bucle for que prueba si hay instancia de gato a medida que lo recorre, si es así, aumenta en "1" la variable total, accedo a Gato para poder u
// "test_leucemia", así puedo hacer seguimiento del segundo atributo de la función, que se emplea para contar aquellos gatos que SI tienen el test. Y los muestro en consola
public void mostrarEstadisticasGatos() {
    try {
        int total = 0;
        int conLeucemia = 0;

        for (Animal a : animals) {
            if (a instanceof Gato) {
                total++;
                Gato g = (Gato) a;
                if (g.testLeucemia) {
                    conLeucemia++;
                }
            }
        }

        System.out.println("Número total de gatos: " + total);
    } catch (Exception e) {
        System.out.println("Excepción: " + e.getMessage());
    }
}

```

```

        System.out.println("Número total de gatos: " + total);
        System.out.println("Número de gatos con test de leucemia positivo: " + conLeucemia);
    } catch (Exception e) {
        System.out.println("Excepción: " + e.getMessage());
    }
}

// Esta función recorre la instancia de animal y comprueba si coinciden los chips
public Animal buscarPorChip(int chip) {
    try {
        for (Animal a : animals) {
            if (a.numeroChip == chip) {
                return a;
            }
        }
        return null;
    } catch (Exception e) {
        System.out.println("Excepción: " + e.getMessage());
        return null;
    }
}

// Esta función devuelve el animal que tenga el chip introducido como parámetro, y sus datos
public void mostrarAnimals(int chip) {
    try {
        boolean encontrado = false;

        for (Animal animal : animals) {
            if (animal.numeroChip == chip) {
                animal.mostrar();
                encontrado = true;
                break;
            }
        }

        if (!encontrado) {
            System.out.println("No se ha encontrado ningún animal con el chip: " + chip);
        }
    } catch (Exception e) {
        System.out.println("Excepción: " + e.getMessage());
    }
}
}

```

import java.util.*; // Importo los utils de java

public class Main {

Scanner sc = **new** Scanner (System.in); // Creo la instancia del scanner

ArrayList <Animal> animals = **new** ArrayList<>(); // Y el arraylist donde depositaré los objetos de la clase padre Animal

public static void main (String[]args) { // En main creo la instancia de la clase para realizar las operaciones de los métodos

```
    Main hito1 = new Main();  
    hito1.menu();  
}
```

public void menu () {

try { // Try catch para añadir una corrección de errores en el caso de que no funcione correctamente o de error (aplica el mismo comentario en los otros try catch)

int opcion;

do { // Me aseguro de que al menos una vez muestre todas las opciones pues será necesario para interactuar con el programa, hasta que la opción seleccionada sea la 4, salir

System.**out**.println(" !Bienvenido!! \n Podemos a su disposicion las siguientes interacciones: \n Posibles elecciones; \n -----
-----");

System.**out**.println("1 - Agregar perro \n 2 - Agregar gato \n -3 Listar animales \n - 4 Buscar animal \n - 5 Realizar adopcion \n - 6 Dar de baja animal \n - 7 Mostrar estadísticas de gatos \n - 8 Salir ");

opcion = sc.nextInt();

// Cada caso es una opción dentro del programa, en la tercera listo todos los animales, en la cuarta a través del scanner pido el chip y llama a la función para buscar según el chip encontrado dentro del array.

//La quinta me permite adoptar un animal que con los atributos de la instancia de Adopcion. La sexta elimina el registro del animal pidiendo el chip o id.

//La septima me muestra el total de gatos adoptados y el total de tests que tienen. La octava opción permite salir del bucle while, mientras que si se da cualquier otra se señala como no válida, puesto que no encierra función alguna.

```
switch(opcion) {
```

```
case 1:
```

```
    añadirPerro();
```

```
    break;
```

```
case 2:
```

```
    añadirGato();
```

```
    break;
```

```
case 3:
```

```
    listarAnimales();
```

```
    break;
```

```
case 4:
```

```
    System.out.println("Dame el número de chip  
para mostrar sus datos ");
```

```
    int chip = sc.nextInt();
```

```
    mostrarAnimals(chip);
```

```
    break;
```

```
case 5:
```

```

        realizarAdopcion();
        break;

    case 6:
        darDeBajaAnimal();
        break;

    case 7:

        mostrarEstadisticasGatos();
        break;

    case 8:

        System.out.println("Saliendo del programa...");
        break;

    default:
        System.out.println("No es opcion válida");
    }
    System.out.println("La elección anterior fue "+opcion);
}

while (opcion != 8);

```

```

    }catch(Exception e) {
        System.out.println("Excepcion"+e);
    }

}

```

// En esta función (dentro de catch try por los posibles errores), establezco que el número de chip sea el tamaño del array +1, lo que evita repeticiones de id, así como que sea una adición incremental de este valor.

// Poco a poco se piden los valores de los atributos correspondientes que se añadirán a la nueva instancia de perro con los valores coincidentes del constructor

// En adición creo una instancia de Adopcion para definir si esta adoptado o no, en el caso de que lo este, entonces pido los datos para luego, en la adición del perro, o del gato correspondiente, poder tener un objeto de Adopcion como parte del array.

```

public void añadirPerro() {

    try {

        int num_chip = numAnimales()+1;

        sc.nextLine();

        System.out.println("Introduce nombre.");

        String nombre = sc.nextLine();

        System.out.println("Introduce edad.");
        int edad = sc.nextInt();

        sc.nextLine();

        System.out.println("Introduce raza.");
    }
}

```

```

String raza = sc.nextLine();

Adopcion adopcion = null;

System.out.println("es adoptado?.");
boolean esAdoptado = sc.nextBoolean();

sc.nextLine();

if (esAdoptado) {
System.out.println("Nombre del adoptante:");
String nombreAdoptante = sc.nextLine();

System.out.println("Teléfono del adoptante:");
String telefono = sc.nextLine();

System.out.println("Fecha de adopción (DD/MM/YYYY):");
String fecha = sc.nextLine();

adopcion = new Adopcion(nombreAdoptante, telefono, fecha);
}

System.out.println("Introduce tamaño.");
String tamaño = sc.nextLine();

animals.add(new
Perro(num_chip,nombre,edad,raza,adopcion,tamaño));
} catch (Exception e) {
System.out.println("Excepcion"+e);
}

```

}

// En esta función (dentro de catch try por los posibles errores), establezco que el número de chip sea el tamaño del array +1, lo que evita repeticiones de id, así como que sea una adición incremental de este valor.

// Poco a poco se piden los valores de los atributos correspondientes que se añadirán a la nueva instancia de perro con los valores coincidentes del constructor

// En adición creo una instancia de Adopcion para definir si esta adoptado o no, en el caso de que lo este, entonces pido los datos para luego, en la adición del perro, o del gato correspondiente, poder tener un objeto de Adopcion como parte del array.

```
public void añadirGato() {
```

```
    try {
```

```
        int num_chip = numAnimales()+1;
```

```
        System.out.println("Introduce nombre.");
```

```
        sc.nextLine();
```

```
        String nombre = sc.nextLine();
```

```
        System.out.println("Introduce edad.");
```

```
        int edad = sc.nextInt();
```

```
        System.out.println("Introduce raza.");
```

```
        String raza = sc.nextLine();
```

```
        sc.nextLine();
```

```
        Adopcion adopcion = null;
```

```
        System.out.println("es adoptado?.");
```

```
        boolean esAdoptado = sc.nextBoolean();
```

```

        sc.nextLine();

        if (esAdoptado) {
            System.out.println("Nombre del adoptante:");
            String nombreAdoptante = sc.nextLine();

            System.out.println("Teléfono del adoptante:");
            String telefono = sc.nextLine();

            System.out.println("Fecha de adopción (DD/MM/YYYY):");
            String fecha = sc.nextLine();

            adopcion = new Adopcion(nombreAdoptante, telefono, fecha);
        }

        System.out.println("Tiene test leucemia?.");
        boolean test_leucemia = sc.nextBoolean();

        animals.add(new
Gato(num_chip,nombre,edad,raza,adopcion,test_leucemia));
    }catch(Exception e) {
        System.out.println("Excepcion"+e);
    }

}

// Esta es la función utilizada para obtener el tamaño del arrayList, en base a lo cual
marco cuál tendría que ser el valor del array. Una variable en un bucle for recorre el array, y su
valor aumenta en "1" hasta que lleva al valor del tamaño del array

public int numAnimales() {
    int no = 0;

```

```

        for (Animal animal : animals) {

            no ++;

        }

        return no;

    }

```

// Esta función utiliza un bucle for para recorrer el array, en busca del mismo valor de chip que se introduce como parámetro, en el momento en el que el valor "numero_chip", coincide con el de "chip" pasado como parámetro,

// utiliza la función mostrar(), presente en ambas clases, para entregar los datos del animal

```

public void listarAnimales() {

    try {

        System.out.println("Listado de todos los animales:\n -----
        -----\\n");

        for (Animal a : animals) {

            a.mostrar();

            System.out.println("-----");

        }

        }catch(Exception e) {

            System.out.println("Excepcion"+e);

        }

    }

```

//Esta funcion pide los datos del animal, en el caso de que el chip del mismo no se encuentre, devuelve mensaje de error por no estar registrado, si ya existe la instancia de adopcion para ese animal,

// devuelve que ya esta adoptado, de lo contrario, solicita los datos correspondientes

```

public void realizarAdopcion() {

    try {

        System.out.println("Introduce el número de chip del animal que será adoptado:");

        int chip = sc.nextInt();

```

```
sc.nextLine();
```

abajo //Aquí uso la función buscarPorChip para comprobar que exista, definida mas

```
Animal animal = buscarPorChip(chip);
```

```
if (animal == null) {
```

```
    System.out.println("El animal no está registrado.");
```

```
    return;
```

```
}else if (animal.adopcion != null) {
```

```
    System.out.println("Este animal ya ha sido adoptado.");
```

```
} else {
```

```
    System.out.println("Nombre del adoptante:");
```

```
    String nombre = sc.nextLine();
```

```
    System.out.println("Teléfono del adoptante:");
```

```
    String telefono = sc.nextLine();
```

```
    System.out.println("Fecha de adopción (DD/MM/YYYY):");
```

```
    String fecha = sc.nextLine();
```

```
    animal.adopcion = new Adopcion(nombre, telefono, fecha);
```

```
    System.out.println("¡Adopción registrada con éxito!");
```

```
}
```

```
} catch (Exception e) {
```

```
    System.out.println("Excepción: " + e.getMessage());
```

```
}
```

```
}
```

// Esta funcion pide el chip y si no existe la instancia con ese valor de chip devuelve que no esta registrado, de lo contrario, si es !null es decir, si existe, entonces lo elimina.

//Antes de esto, si resulta que ademas estaba adoptado, me aseguro de eliminar primero la instancia, dejando el objeto y los datos a null, antes de eliminarlo del todo

```
public void darDeBajaAnimal() {  
    try {  
        System.out.println("Introduce el número de chip del animal para darle baja:");  
        int chip = sc.nextInt();  
        sc.nextLine();  
  
        Animal animal = buscarPorChip(chip);  
        if (animal == null) {  
            System.out.println("El animal no está registrado.");  
            return;  
        } else {  
            if (animal.adopcion != null) {  
                System.out.println("El animal estaba adoptado, elimino datos");  
                animal.adopcion = null;  
            }  
            animals.remove(animal);  
            System.out.println("Animal dado de baja con éxito.");  
        }  
  
    } catch (Exception e) {  
        System.out.println("Excepción: " + e.getMessage());  
    }  
}
```

// Muestro las estadísticas de los gatos con un bucle for que prueba si hay instancia de gato a medida que lo recorre, si es así, aumenta en "1" la variable total, accedo a Gato para poder utilizar el atributo;

// "test_leucemia", así puedo hacer seguimiento del segundo atributo de la función, que se emplea para contar aquellos gatos que SI tienen el test. Y los muestro en consola

```
public void mostrarEstadisticasGatos() {
```

```
    try {
```

```
        int total = 0;
```

```
        int conLeucemia = 0;
```

```
        for (Animal a : animals) {
```

```
            if (a instanceof Gato) {
```

```
                total++;
```

```
                Gato g = (Gato) a;
```

```
                if (g.test_leucemia) {
```

```
                    conLeucemia++;
```

```
                }
```

```
            }
```

```
        }
```

```
        System.out.println("Número total de gatos: " + total);
```

```
        System.out.println("Número de gatos con test de leucemia positivo: " +  
conLeucemia);
```

```
    }catch (Exception e) {
```

```
        System.out.println("Excepción: " + e.getMessage());
```

```
    }
```

```
}
```

// Esta función recorre la instancia de animal y comprueba si coinciden los chips

```
public Animal buscarPorChip(int chip) {
```

```
    try {
```

```

    for (Animal a : animals) {
        if (a.numero_chip == chip) {
            return a;
        }
    }
    return null;
} catch (Exception e) {
    System.out.println("Excepción: " + e.getMessage());
    return null;
}
}

```

// Esta funcion devuelve el animal que tenga el chip introducido como parametro, y sus datos

```

public void mostrarAnimals(int chip) {
    try {
        boolean encontrado = false;

        for (Animal animal : animals) {
            if (animal.numero_chip == chip) {
                animal.mostrar();
                encontrado = true;
                break;
            }
        }

        if (!encontrado) {
            System.out.println("No se ha encontrado ningún animal con el chip: " + chip);
        }
    }
}

```

```

    } catch (Exception e) {
        System.out.println("Excepción: " + e.getMessage());
    }
}
}

```

Funcionamiento código

Añadir perro

```

1
Introduce nombre.
asd
Introduce edad.
23
Introduce raza.
raza
es adoptado?.
true
Nombre del adoptante:
pol
Teléfono del adoptante:
5612612
Fecha de adopción (DD/MM/YYYY):
09/04/2003
Introduce tamaño.
mediano
La elección anterior fue 1

```

Añadir gato

```
Introduce nombre.  
Lora  
Introduce edad.  
32  
Introduce raza.  
blanco  
es adoptado?.  
true  
Nombre del adoptante:  
pol  
Teléfono del adoptante:  
125125  
Fecha de adopción (DD/MM/YYYY):  
05/12/2025  
Tiene test leucemia?.  
false  
La elección anterior fue 2
```

Buscar Animal

```
1  
Número de chip del perro : 1  
Nombre ; asd  
edad: 23de raza: raza  
De tamaño:mediano  
Adoptado por:  
pol, Con el DNI:  
5612612, Fecha en la que fue adoptado:  
09/04/2003  
La elección anterior fue 4  
!Bienvenido!!
```

Busco el primero, perro

```
1  
Número de chip del perro : 1  
Nombre ; asd  
edad: 23de raza: raza  
De tamaño:mediano  
Adoptado por:  
pol, Con el DNI:  
5612612, Fecha en la que fue adoptado:  
09/04/2003  
La elección anterior fue 4  
!Bienvenido!!
```

Busco el segundo, gato

```
Número de chip del gato : 2
Nombre ; Lua
edad: 12
de raza:
  Tiene el test de leucemia?true
No está adoptado.

La elección anterior fue 3
!Bienvenido!!
```

Listado de Animales

```
3
Listado de todos los animales:

Número de chip del perro : 1
Nombre ; asd
edad: 23de raza: raza
De tamaño:mediano
Adoptado por:
pol, Con el DNI:
5612612, Fecha en la que fue adoptado:
09/04/2003

Número de chip del gato : 2
Nombre ; Lua
edad: 12
de raza:
  Tiene el test de leucemia?true
No está adoptado.

La elección anterior fue 3
!Bienvenido!!
```

5 Adoptar

```

Introduce el número de chip del animal que será adoptado:
2
Nombre del adoptante:
Nico
Teléfono del adoptante:
978236598123
Fecha de adopción (DD/MM/YYYY):
09/04/2025
¡Adopción registrada con éxito!
La elección anterior fue 5
!Bienvenido!!
Podemos a su disposicion las siguientes interacciones:
Posibles elecciones:

```

6 Dar de baja

```

1 - Agregar perro
2 - Agregar gato
-3 Listar animales
- 4 Buscar animal
- 5 Realizar adopcion
- 6 Dar de baja animal
- 7 Mostrar estadisticas de gatos
- 8 Salir
6
Introduce el número de chip del animal para darle baja:
2
El animal estaba adoptado, elimino datos
Animal dado de baja con éxito.
La elección anterior fue 6
!Bienvenido!!
Podemos a su disposicion las siguientes interacciones:
Posibles elecciones:

```

Y no aparece Lua

```

Número de chip del perro : 1
Nombre ; asd
edad: 23de raza: raza
De tamaño:mediano
Adoptado por:
pol, Con el DNI:
5612612, Fecha en la que fue adoptado:
09/04/2003

La elección anterior fue 3
!Bienvenido!!
Podemos a su disposicion las siguientes in

```

7 motrar estadísticas

```
Número total de gatos: 0
Número de gatos con test de leucemia positivo: 0
La elección anterior fue 7
```

Porque he eliminado al gato

```
1 - Agregar perro
2 - Agregar gato
-3 Listar animales
- 4 Buscar animal
- 5 Realizar adopcion
- 6 Dar de baja animal
- 7 Mostrar estadisticas de gatos
- 8 Salir
2
Introduce nombre.
asd
Introduce edad.
4
Introduce raza.
loco
es adoptado?.
false
Tiene test leucemia?.
true
La elección anterior fue 2
!Bienvenido!!
Podemos a su disposicion las siguientes interacciones:
Posibles elecciones;

1 - Agregar perro
2 - Agregar gato
-3 Listar animales
- 4 Buscar animal
- 5 Realizar adopcion
- 6 Dar de baja animal
- 7 Mostrar estadisticas de gatos
- 8 Salir
7
Número total de gatos: 1
Número de gatos con test de leucemia positivo: 1
```

Volví a añadir a uno y aparecia

Salir del programa

```
7. Mostrar estadísticas de g  
- 8 Salir  
8  
Saliendo del programa ...  
La elección anterior fue 8
```

Enlace a GitHub

<https://github.com/Cortes-cmd/Programacion.git>

